

Computer Science COMP-3220 – Summer 2026

Assignment 2

Due: End of Sunday, July 19, 2026

Note: Submit your assignment as a **single pdf file** on Brightspace. The pdf file should be named Assignment2_StudentId.pdf, which replaces *StudentId* with your university student number.

Total Mark: 40

Question 1 — Factory Pattern (13 marks)

Extending an Employee Management System

You are given the following Java code:

```
class EmployeeManager {  
  
    public Employee createEmployee(String type) {  
  
        if (type.equals("FULLTIME")) {  
            return new FullTimeEmployee();  
        }  
        else if (type.equals("PARTTIME")) {  
            return new PartTimeEmployee();  
        }  
        else if (type.equals("CONTRACT")) {  
            return new ContractEmployee();  
        }  
  
        return null;  
    }  
}
```

Suppose the company plans to add additional employee types such as:

- InternEmployee
- TemporaryEmployee
- RemoteEmployee

Tasks:

1. Identify at least two design issues in the given code. (3 marks)
2. Refactor the design using the **Factory Pattern**. (5 marks)

Complete the design by:

- Creating an `Employee` interface
 - Implementing at least three concrete employee classes
 - Creating an `EmployeeFactory`
 - Updating the client code to use the factory
3. Briefly explain how the Factory Pattern improves: (2 marks)

- Extensibility
- Maintainability
- 4. Draw a UML Class Diagram for your solution. Include: (3 marks)
 - Employee interface
 - Concrete employee classes
 - EmployeeFactory
 - Relationships

Question 2 — Singleton Pattern (13 marks)

Designing a Configuration Manager

A software system should maintain only one global configuration object. You are given the following code:

```
class ConfigurationManager {

    private String databaseURL;

    public ConfigurationManager() {
        databaseURL = "localhost";
    }

    public String getDatabaseURL() {
        return databaseURL;
    }

}
```

The system currently allows unlimited instances:

```
ConfigurationManager c1 =
    new ConfigurationManager();

ConfigurationManager c2 =
    new ConfigurationManager();
```

Tasks:

1. Explain why allowing multiple configuration managers may cause problems. Give at least two reasons. (3 marks)
2. Convert the class into a proper **Singleton Pattern** implementation. (5 marks)

Complete the design by:

- Making the constructor private
- Creating a static instance variable
- Providing a public access method
- 3. Explain two advantages of using the Singleton Pattern in this scenario. (2 marks)
- 4. Draw a UML Class Diagram for the Singleton implementation. Include: (3 marks)
 - Singleton class
 - Private constructor
 - Static instance
 - Static getInstance()

Question 3 — Command Pattern (14 marks)

Implementing a Smart Home Remote Control

You are given a partially implemented smart-home system.

```
class Light {  
  
    public void turnOn() {  
        System.out.println("Light ON");  
    }  
  
    public void turnOff() {  
        System.out.println("Light OFF");  
    }  
}  
  
class RemoteControl {  
  
    public void pressButton(String action) {  
  
        if(action.equals("ON")) {  
            // turn on light  
        }  
        else if(action.equals("OFF")) {  
            // turn off light  
        }  
    }  
}
```

The system will later support:

- Fan
- Garage Door
- Stereo System

Tasks:

1. Explain why the current design is not flexible. Identify at least two design issues. (3 marks)
2. Refactor the system using the **Command Pattern**. (6 marks)

Complete the design by:

- Creating a Command interface
 - Implementing:
 - LightOnCommand
 - LightOffCommand
 - Updating RemoteControl to use commands
 - Associating commands with a Light receiver
3. Explain how the Command Pattern improves extensibility and reduces coupling. (2 marks)
 4. Draw a UML Class Diagram for the solution. Include: (3 marks)
 - Command interface
 - Concrete commands
 - Receiver (Light)
 - Invoker (RemoteControl)